```
      bubbleUp(pairs, index)
                  #4
    elsif (newPriority > oldPriority) then
      pushDown(pairs, index)
                  #5
```

#1 Get the position of the element to update
#2 Check if the element is actually stored in the heap, and update it with the new value/priority
#3 Check if the new priority is higher than the old one
#4 If so, bubble up the element towards the root
#5 Otherwise, push it down towards the leaves

From a performance point of view, the challenge of this method is in line #1: the search for the old element can take linear time, because on a failed search (when the element is not actually in the heap) we will have to search the whole heap.
To improve this worst case, it is possible to use auxiliary data structures that will help us in performing search more efficiently. For instance, we can store a `map` associating each element in the heap with its index: if implemented with an hash-table, the lookup would only need an amortized `O(1)` time.
We will see this in further detail when describing the `contains` method. For now, let's just remark that if the `find` operation takes at most logarithmic time, the whole method is also logarithmic.

As we saw, the first case can always be implemented more efficiently[13] and luckily for us, most algorithms only require us to reduce elements priority.